Task 2

Mentor Review Packet

Remediating Broken Object Level Authorization (BOLA/IDOR)

Prepared for Datheon internship mentors and project reviewers

05 November 2026

Review scope and safety notice

This packet documents a loopback-only training project using five fictional records. It is not a production assessment, and its fixed session tokens are public lab fixtures rather than real credentials.

Contents

1 Reviewer at a glance 1
2 Scope and review boundary 1
3 Authorization workflow 1
4 Reproduction results 1
5 Security finding analysis 2
5.1 Before and after 2
5.2 Expected production pattern 2
6 Visual evidence for reviewers 2
7 Verification summary 3
8 Mentor review conclusion 4

1 Reviewer at a glance

This packet gives a mentor a concise, reproducible view of the Task 2 BOLA/IDOR remediation. The project starts from an intentionally vulnerable single-identifier lookup and replaces it with a protected Express route that derives the user identity from server-side session context and checks both the requested record ID and authenticated owner.

Verified implementation The local integration suite passed **15 tests** and the TypeScript check passed. The tests cover missing or invalid sessions, owner access for all five demo users, cross-user denial with generic `404`, non-disclosure, browser routing, and the page manifest.

What reviewers can verify Open the embedded browser capture to see Student A's normal dashboard. Then compare the selected cURL results: no session gives `401`, Student A reading application `104` gives `200`, and the same Student A session requesting Student B's `105` gives a generic `404` without record data.

2 Scope and review boundary

Item	Value
Target	Northstar Student Portal BOLA Guard Lab
Base URL	`http://127.0.0.1:3000`
Protected endpoint	`GET /api/records/:recordId`
Data	Five fictional in-memory records, IDs 104–108
Authentication	Fixed public training tokens mapped server-side for the lab
Out of scope	Production Datheon systems, real users, deployment, database, or external hosts

3 Authorization workflow

1. Authenticate

Require a recognized session token.

→

2. Resolve owner

Derive the principal from the server-side session map.

→

3. Authorize

Query by both record ID and authenticated owner ID.

The critical security property is that a client-supplied record identifier is not sufficient. The secure route establishes **who** is asking first, then enforces ownership in the server-side lookup.

4 Reproduction results

Representative local cURL checks

```
curl -i http://127.0.0.1:3000/api/records/104
# HTTP/1.1 401 Unauthorized

curl -i -H 'Authorization: Bearer lab-session-user-a' \\
  http://127.0.0.1:3000/api/records/104
# HTTP/1.1 200 OK — Student A's own synthetic record

curl -i -H 'Authorization: Bearer lab-session-user-a' \\
  http://127.0.0.1:3000/api/records/105
# HTTP/1.1 404 Not Found — generic body, no Student B data
```

Scenario	Status	Reviewer observation
No session ⟶ record 104	401	Authentication is required
Student A ⟶ own record 104	200	Owner access succeeds
Student A ⟶ Student B record 105	404	No record or internal detail is disclosed
Unknown or malformed ID	404	Generic failure behavior

5 Security finding analysis

5.1 Before and after

The vulnerable example looks up an application using only a client-controlled ID. The remediation adds a session guard, derives the authenticated user on the server, and performs an ownership-constrained lookup. The successful response also omits internal `ownerId` data.

Privacy-preserving denial A non-owned record and an unknown record use the same generic `404` behavior. This reduces account and identifier enumeration and avoids exposing database-style details.

5.2 Expected production pattern

A database-backed production query should constrain both values, for example `WHERE application_id = ? AND owner_id = ?`, using a validated session principal. The service should not fetch by ID first and rely on a later client-side or middleware-only check.

6 Visual evidence for reviewers

The browser capture below shows the protected Student A portal loaded at `appId=104`. The UI presents only the current account's dashboard and identifies the environment as local training data; it does not expose an account picker or object lookup form.

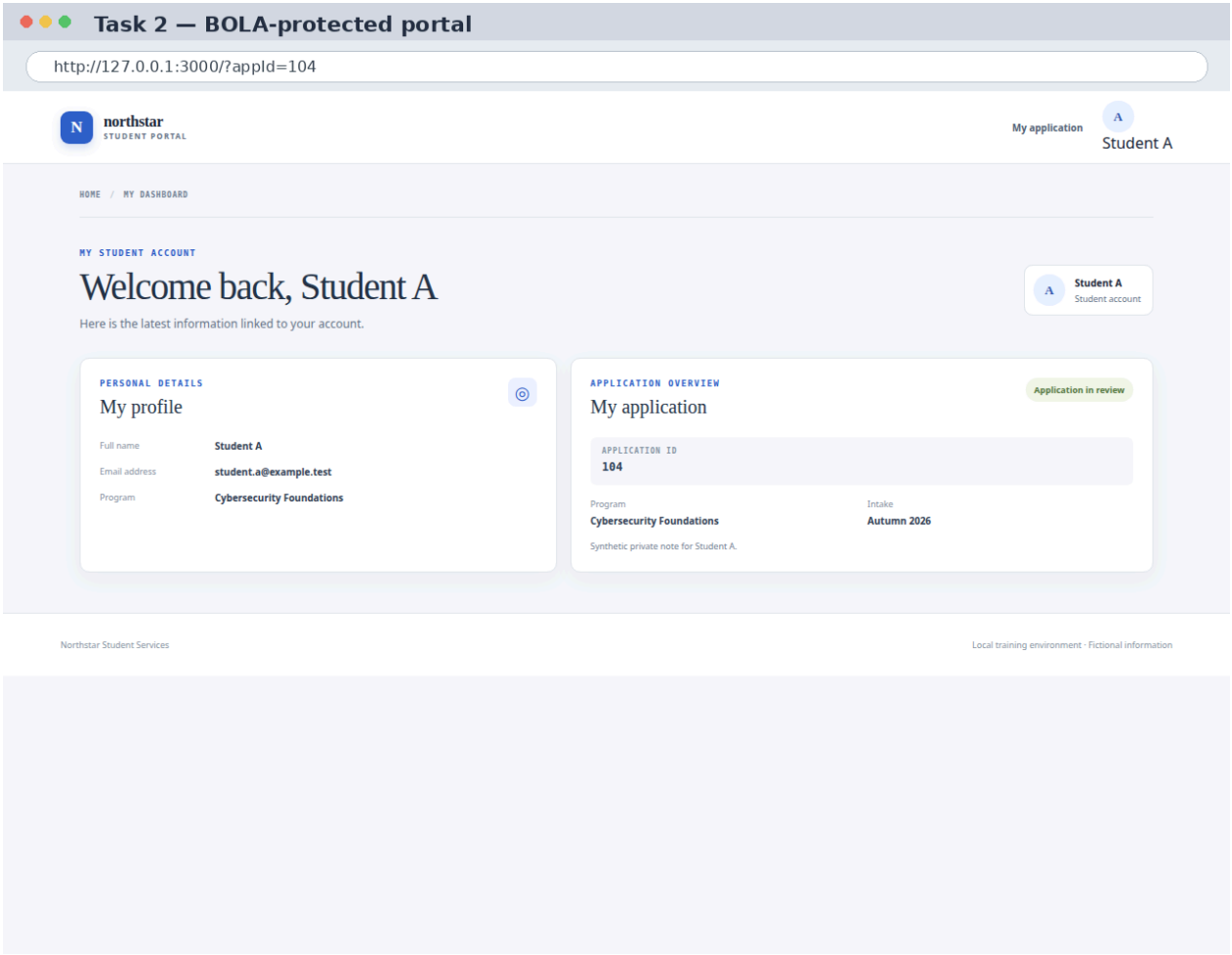


图 1 Browser view of the protected Student A dashboard at `127.0.0.1:3000/?appId=104`.

Visual evidence interpretation Changing only `104` to `105` in the same browser session leads to the ordinary "We couldn't find that application" page. The underlying API returns the same generic `404` for non-owned and unknown records; Student B's record is not rendered.

7 Verification summary

Check	Result	Coverage
Vitest/Supertest integration suite	15 passed	Authentication, owner access, denial, non-disclosure, browser shell
TypeScript check	Passed	Strict compile-time verification
Training boundary	Pass	Loopback-only, fictional data, no production integration

8 Mentor review conclusion

Task 2 demonstrates a complete BOLA/IDOR remediation rather than a UI-only workaround. The route rejects unauthenticated requests, binds access to the server-side session identity, prevents cross-user disclosure, and proves those properties with automated tests. The implementation is suitable for review as a local training deliverable; it is not a production authentication system.

Documentation note: This packet was assembled with AI assistance from the supplied Task 2 source files. All displayed records and tokens are fictional local training fixtures.